PHASES

0–18

Turk-OS

Roadmap: from zero to a working OS

A 19-phase study and build plan for a UNIX-like 32-bit x86 kernel and its RISC-V and ARM ports, based on seven operating-system books

Overall progress toward Turk-OS 1.0 (this phase in red) 0 % → 100 %

Contents

1 How to use this plan **1**
- 1.1 The study loop . . . 2
- 1.2 Time budget . . . 2

2 Your seven books and how each one is used **2**
- 2.1 Getting your copies . . . 2
- 2.2 References outside the seven books . . . 3

3 Key technical decisions **4**

4 The phase map **7**

5 Phase summary **8**

6 Timeline **9**

7 What Turk-OS 1.0 and 2.0 look like **9**

8 Risks, and what to do when you are stuck **11**

9 Master milestone tracker **12**

1 How to use this plan

You know basic C, you have never written assembly, and you have never built an operating system. That is a normal starting point: the authors of LOB were in a similar position when they wrote it. This plan takes you from an empty folder to **Turk-OS 1.0**, a small UNIX-like operating system for 32-bit x86 PCs. It boots with GRUB, runs preemptive multitasking with separate address spaces, loads ELF programs from its own on-disk file system, and drops you into a shell with pipes and redirection. Everything runs in QEMU first. Real hardware is optional. A fifth stage then ports Turk-OS to two more architectures, 32-bit RISC-V and 64-bit ARM, and ends at **Turk-OS 2.0**: one source tree whose kernel and userland boot on all three.

The route has nineteen phases, numbered 0 to 18, in five stages. Each phase has its own PDF in `docs/pdf/` and ends with a concrete, testable milestone. The percentages in the headers of Phases 0–15 ("0 % → 5 %") estimate how much of the road to Turk-OS 1.0 is behind you when that milestone is met. Phase 15 ends at 100 %, tags version 1.0, and also lists optional tracks for going further. The three Stage V documents (Phases 16–18) show a second bar, *Stage V progress toward Turk-OS 2.0*,

on its own scale: Phase 16 takes it from 0 % to 20 %, Phase 17 from 20 % to 60 %, and Phase 18 from 60 % to 100 %.

Every phase document follows the same layout, so you always know where to look. An *at a glance* box gives the duration, difficulty, prerequisites and result. *Why this phase* explains where it fits. *Concepts* explains the ideas you must understand before you write code. The *reading plan* lists book chapters with printed page numbers, each marked REQUIRED, RECOMMENDED or OPTIONAL. *Build it* breaks the work into numbered tasks with starter snippets and a way to verify each one. The document ends with a debugging table, the milestone checklist, self-check questions, and stretch goals. The Stage V documents also cite free specifications and the xv6 book; their chips are explained in a second legend at the end of those documents, and of this one.

1.1 The study loop

The same loop works in every phase. Keep to it, because skipping the reading makes the coding much slower.

1. **Read the theory first**, usually OSTEP, then the REQUIRED items from the other books.
2. **Read the practical chapter** in LOB (when there is one) to see the concrete x86 steps.
3. **Look at a real system.** The lens box points to the matching OSDI section and MINIX source file, and ends with how Linux does the same job, from LKD and LKP. In Stage V the same box looks at xv6-riscv or Linux instead.
4. **Build in small steps.** One task at a time, boot after each change, commit when it works.
5. **Verify.** Every task has a *Verify* line. A task is not finished until you have checked it.
6. **Write it down.** Keep a lab journal (`docs/journal.md`) with what you tried, what broke and why.
7. **Answer the self-check questions** without looking. If you can't, go back to the reading.

1.2 Time budget

The estimates assume about 10–12 hours a week, which adds up to roughly 43 weeks (about ten months) to Turk-OS 1.0. Stage V adds about 15 weeks, so the whole plan takes about 58 weeks, roughly fourteen months. Some phases will go faster than planned and some much slower; paging (Phase 6) and processes (Phase 8) are where most people get stuck, and in Stage V it is the traps and the virtio disk of the RISC-V port (Phase 17) and the AArch64 MMU (Phase 18). What counts is meeting each phase's exit criteria. Never start a phase while the previous milestone is still shaky, because every later phase builds on top of it.

2 Your seven books and how each one is used

The seven books complement each other. Only LOB tells you what to type. OSTEP, OSC and MOS explain why it works; OSDI shows a complete small kernel, MINIX 3; and the two Linux books, LKD and LKP, show how today's most widely used kernel does each job. None of them comes with this plan, so get your own copies before you start Phase 0; Section 2.1 says where.

2.1 Getting your copies

LOB and OSTEP are free online from their authors. OSC, MOS and OSDI are commercial textbooks: buy them from the publisher or a bookshop (a used copy is fine), or borrow them from a library. Table 2 lists the exact editions that the reading plans cite.

⚠ Page numbers belong to these editions

The page ranges in the reading plans are the *printed* page numbers of the editions in Table 2. They will not match another edition, and a reflowable e-book has no fixed pages at all, so find each reading by the chapter and section numbers that every row gives. For OSTEP this is the

Code	Book	Role in this plan	How to read it
LOB	The little book about OS development (78 pp.)	The practical spine. Its 14 short chapters are the build order for Phases 1–14.	Read each chapter right before its phase. It uses GRUB Legacy and Bochs; the phase docs give the modern QEMU and GRUB 2 equivalents.
OSTEP	Operating Systems: Three Easy Pieces	Main theory text. Short, friendly chapters built around one problem each.	Your first theory read in every phase. Do the homework simulators when a phase suggests it.
OSC	Operating System Concepts, 10th ed.	Encyclopedic reference. Good second explanation and exam-style depth.	Use it when an OSTEP chapter didn't click, or when you want the complete picture of a topic.
MOS	Modern Operating Systems, 4th ed.	Strongest on hardware, I/O, clocks, disks, security and OS design (Ch. 12).	Read the hardware and I/O sections closely; they explain the devices you will program.
OSDI	Operating Systems: Design and Implementation, 3rd ed.	The only book with a complete real x86 kernel: MINIX 3, printed line by line.	After you understand a topic, read how MINIX implements it. Don't copy it: MINIX is a microkernel, Turk-OS is monolithic.
LKD	Linux Kernel Development, 3rd ed.	A compact tour of how Linux implements each subsystem: scheduling, interrupts and bottom halves, synchronisation, memory, the VFS, block I/O.	Read the matching chapter after the theory, as the second real-system lens next to MINIX.
LKP	Linux Kernel Programming (2021)	The newest book in the plan: modern kernel practice for logging, the buddy and slab allocators, the CFS scheduler and synchronisation.	Read it for practices; skip the parts about building Linux and writing loadable modules, which Turk-OS doesn't need (Track G excepted).

Table 1: Roles of the seven books.

normal case: each free chapter PDF starts again at page 1, and the current version is paginated differently from version 0.90. Chapters 1–43, and every section this plan cites, have the same numbers in both versions, and the page range still tells you how long a reading is. One title changed: chapter 23, *The VAX/VMS Virtual Memory System*, is now *Complete Virtual Memory Systems*, with VAX/VMS in §23.1.

2.2 References outside the seven books

Books explain ideas, but writing a kernel also needs exact hardware facts. Keep these four free references at hand, and note that LOB relies on them too. The **Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A** (System Programming Guide) is the definitive source for protection, segmentation, paging, interrupts and the TSS; you will read chapters 2–7 in pieces. The **OSDev Wiki** (`https://wiki.osdev.org`) has practical pages on nearly every device in this plan, plus lists of common mistakes. The **Multiboot Specification 0.6.96** defines the contract between GRUB and your kernel. The **NASM manual** covers assembler syntax. The MIT **xv6** teaching kernel is a good optional companion: it is small and UNIX-like, and OSTEP refers to it.

Stage V leaves x86, so the Intel manual, Multiboot and NASM no longer apply there. Its documents cite four more free references, each with its own chip. XV6 is the xv6 book, RISC-V edition, read together with the `xv6-riscv` source: the closest small kernel to what you build in Phase 17. RISCV is the RISC-V Instruction Set Manual, Volume I (unprivileged) and Volume II (privileged architecture),

Code	Edition cited in this plan	Where to get it
LOB	Erik Helin, Adam Renberg. *The little book about OS development*, 2015.	Free at `https://littleosbook.github.io`, as a web page or as a PDF (the *PDF version* link). Page numbers refer to the PDF.
OSTEP	Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, version 0.90, Arpaci-Dusseau Books, 2015.	Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`, one PDF per chapter, in the current version (1.10). Printed copies are sold on the same page.
OSC	Abraham Silberschatz, Peter Baer Galvin, Greg Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. ISBN 978-1-119-32091-3.	Wiley, a bookshop or a library.
MOS	Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. ISBN 978-0-13-359162-0.	Pearson, a bookshop or a library.
OSDI	Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. ISBN 978-0-13-142938-3.	A bookshop (often second-hand) or a library.
LKD	Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. ISBN 978-0-672-32946-3.	A bookshop or a library.
LKP	Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. ISBN 978-1-78995-343-5.	Packt, a bookshop or a library.

Table 2: The editions this plan cites, and where to get them.

plus the RISC-V Supervisor Binary Interface (SBI) specification. ARM is the Arm Architecture Reference Manual for A-profile (DDI 0487) together with Arm's shorter *Learn the architecture* guides. SPEC covers the Devicetree and Virtio specifications and QEMU's documentation for its `virt` boards. The port legend at the end of this document gives the full titles and links.

3 Key technical decisions

These choices hold for the whole plan. The first eight follow from the books, which keeps your reading and your code in step. The last five add the ports of Stage V and follow from the free specifications instead.

Decision	Choice	Why
Target CPU	IA-32 (i386), 32-bit protected mode, BIOS boot	LOB, OSDI (MINIX 3) and the Intel examples in OSC §9.6 are all 32-bit x86. Two-level paging is the simplest real MMU to learn. A 64-bit x86 port stays a Phase 15 track; the RISC-V and ARM ports are Stage V.
Boot loader	GRUB via Multiboot 1; `qemu -kernel` for quick runs	Writing a boot loader is a detour. You write one 512-byte boot sector in Phase 1 to learn how booting works, then let GRUB do the job.
Languages	C (gnu11) plus NASM (Intel syntax)	Assembly only where C cannot work: the entry point, descriptor table loads, interrupt stubs, context switch and entering user mode. That is under 5 % of the code.
Toolchain	`i686-elf` cross GCC and binutils, GNU Make, Git	A cross compiler never assumes a Linux target, which prevents a family of confusing bugs (Phase 0).

Decision	Choice	Why
Emulator, debugger	QEMU and GDB (remote stub); Bochs is optional	QEMU is fast, scriptable, and has a monitor for inspecting registers and page tables. LOB uses Bochs, whose debugger is still handy for descriptor-table bugs.
Kernel structure	Monolithic, modular source tree	Simplest to get right for a first kernel. MINIX's microkernel is studied as a contrast, with an optional experiment in Phase 15.
On-disk file system	MINIX V3 layout with 1 KiB blocks, named TurkFS	The format described in OSDI §5.6, with source. Linux's `mkfs.minix -3` and `fsck.minix` create and check your images, and Linux can even mount them.
Testing	In-kernel `ktest` suites, headless QEMU, `isa-debug-exit`; host tests and parser fuzzing under AddressSanitizer and UBSan; continuous integration	`make test` boots, runs every test and exits with a pass or fail code, so regressions show up at once (Phase 7). CI runs it on every push; negative tests ("evil programs") and a system call fuzzer check the protection boundary from Phases 10 and 11 on.
Port targets	RV32 (`rv32imac`) with Sv32 paging on QEMU `virt`, then optionally your own Timur-RV32IMC core; AArch64 (ARMv8-A) at EL1 with 39-bit virtual addresses on QEMU `virt`, then optionally a Raspberry Pi 4	Sv32 is two-level 4 KiB paging like x86, so the RISC-V kernel can stay at `0xC0000000`. AArch64 then changes the instruction set and the word size together: 4 KiB pages, three-level tables, the kernel in the `TTBR1` high half (Phases 17, 18).
Port firmware and discovery	OpenSBI in M-mode (the kernel runs in S-mode); QEMU or the Pi firmware on ARM; a flattened devicetree instead of Multiboot	The firmware hands over a devicetree that says where RAM and devices are. `lib/fdt.c` reads it into `struct bootinfo`, so generic code never knows which firmware booted it (Phase 16).
System calls on the ports	The same Linux i386 numbers everywhere; `ecall` with the number in `a7`, `svc #0` with it in `x8`	One table and one libc for three architectures; only the trap instruction and the registers change. Linux itself uses a different, generic table on RISC-V and arm64 (`write` is 64 and there is no `fork`), so don't copy numbers from its headers.
Port toolchain	`riscv32-elf` and `aarch64-elf` cross GCC and binutils; GNU `as` for port assembly	Built exactly like `i686-elf` in Phase 0, with the same versions (Task 16.1). NASM only knows x86, so it stays for the i386 code; port assembly lives in preprocessed `.S` files.
Port devices	16550 or PL011 console, PLIC or GICv2, SBI or Arm generic timer, virtio-blk disk, `sifive_test` or semihosting for `make test`	The `virt` boards have no PS/2 keyboard, VGA text screen or IDE disk by default. One virtio-blk driver serves both ports. All three targets are little-endian and TurkFS uses fixed-width fields, so the same disk image works everywhere.

Table 3: Technical decisions for Turk-OS.

Why monolithic, when one of your books is about a microkernel?

In a monolithic kernel, drivers, the file system and the scheduler all run in ring 0 and call each other as plain C functions. In a microkernel like MINIX 3 they run as separate user-space processes that talk by message passing (OSDI §1.5.5, §2.5.1; MOS §1.7). Microkernels isolate faults better, but they force you to build IPC, servers and a protocol for every operation before

anything works. For a first kernel, plain function calls keep the path from idea to running code short. You will still read the MINIX code in every phase, and noticing where the two designs differ is one of the best ways to understand both.

Security starts where its subject starts

Turk-OS is a teaching system, not a production one, and its security goal is concrete: no unprivileged program and no malformed disk image may crash or take over the kernel, and every claim of that kind is backed by a test. Each practice begins in the phase where the thing it protects first exists. Phase 0 writes a short threat-model stub and verifies the toolchain downloads; Phase 2 turns every warning into an error; Phase 6 makes kernel code read-only; Phase 7 adds continuous integration and a build that traps on undefined behaviour; Phase 10 writes the real threat model, routes every user pointer through one small file and starts a table of how each system call checks its arguments; Phase 11 starts the system call fuzzer; Phase 13 treats every on-disk structure as untrusted input. Phase 15 then consolidates: it finishes the audit, adds users and permissions, and ships a release whose build can be reproduced and checked. The model, its trust boundaries and what is out of scope (physical attackers, the boot chain, networks, malicious hardware, side channels) live in `docs/security/threat-model.md`.

When two books disagree

The books were written over fifteen years, from OSDI (2006) to LKP (2021), and sometimes teach different ways to do the same job. The rule: follow the newer book's practice when it is also better for Turk-OS, and keep the older one when it teaches the idea more clearly or when the newer one only makes sense for a production kernel. So Turk-OS **adopts** printk-style log levels and a `dmesg` buffer (Phase 2), top and bottom halves for interrupts (Phase 4), the `0`/`-E` and `ERR_PTR` error convention (Phase 7), saturating `refcount_t` counters and a run-time lock-order check (Phase 9). It **stages** the buddy page allocator after the bitmap (Phase 5) and slab caches under `kmalloc` after the first-fit heap (Phase 7), so you learn the simple version first. It offers a CFS-style **fair scheduler** instead of MLFQ as the optional scheduler (Phase 8). And it **keeps** the MINIX-style buffer cache rather than Linux's page cache (Phase 12), and leaves loadable modules to the optional Track G, because a small monolithic kernel gains little from either.

Why port after 1.0, and why RISC-V before ARM?

The practical books, LOB and OSDI with MINIX, are both x86, and one architecture learned deeply comes first. Porting afterwards is not wasted effort: every place where x86 leaked into generic code shows up the moment you try to compile it for another machine, and fixing those leaks is the best way to learn what is really generic. RISC-V comes first because RV32 keeps the 32-bit word size, Sv32 paging has the same two-level, 4 KiB shape as x86, so the kernel can stay at `0xC0000000`, and the privileged architecture is small enough to read in full. It also leads to your own processor core, Timur-RV32IMC. AArch64 then changes the instruction set and the word size (LP64) at the same time and brings a richer exception and memory model. That is harder, which is why it comes second, and it prepares you for the 64-bit x86 track of Phase 15.

4 The phase map

The nineteen phases fall into five stages. Arrows show the build order. Each phase depends on everything before it. The first four stages lead to Turk-OS 1.0; the fifth ports it and leads to Turk-OS 2.0.

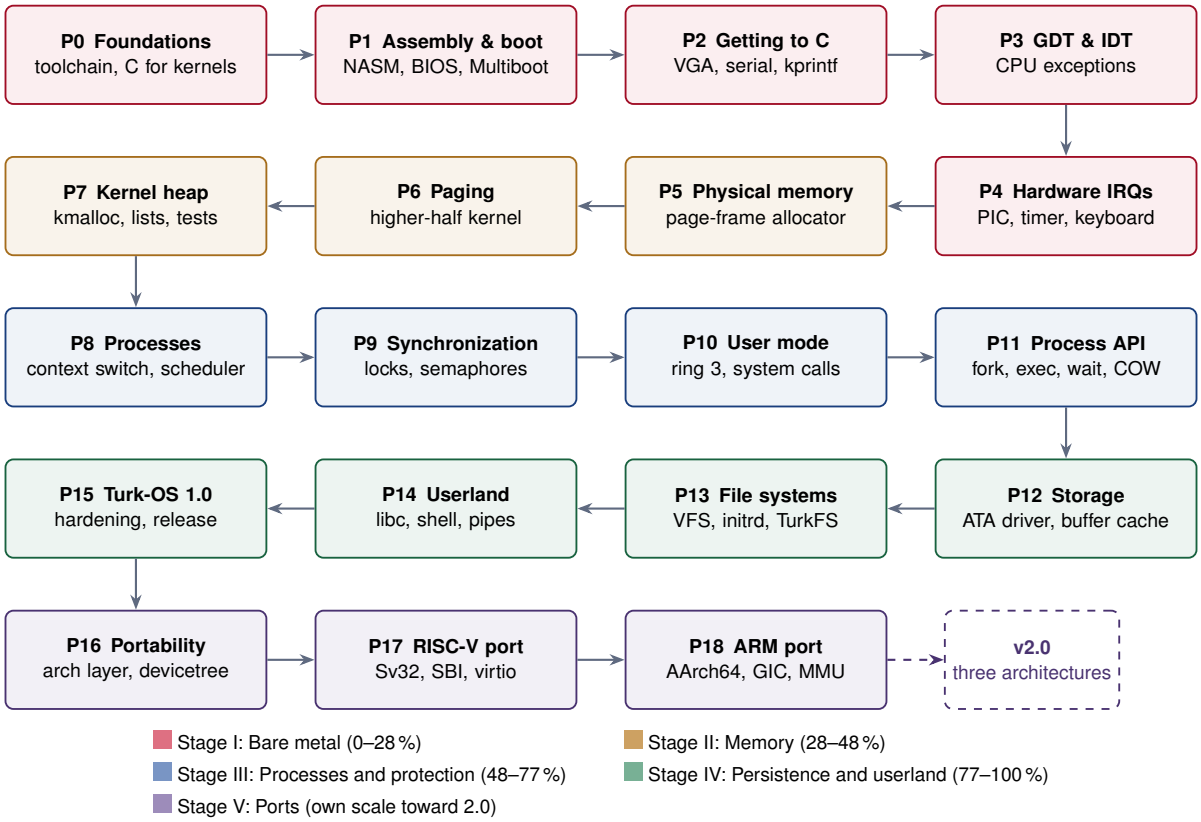


Figure 1: The nineteen phases, grouped into five stages. The percentages of Stages I–IV measure progress toward Turk-OS 1.0.

Stage I, Bare metal (Phases 0–4). You learn the tools, enough assembly to be dangerous, and how a PC boots. By the end the kernel runs C code, prints to the screen and the serial port, survives CPU exceptions with a readable error report, and reacts to the timer and the keyboard through a small built-in command monitor.

Stage II, Memory (Phases 5–7). The kernel learns which RAM exists and hands it out page by page, turns on paging, moves itself to the top gigabyte of the address space, and gets a `kmalloc` heap. You also set up the automated test harness that protects you from regressions for the rest of the project.

Stage III, Processes and protection (Phases 8–11). Multitasking arrives: kernel threads, context switches, a preemptive scheduler, then the locks that make concurrency safe. Programs move to ring 3 with their own address spaces and talk to the kernel through system calls. The stage ends with the UNIX process model: `fork` with copy-on-write, `exec`, `wait` and `exit`.

Stage IV, Persistence and userland (Phases 12–15). A disk driver and buffer cache give you storage, a VFS and TurkFS give you files, and a C library, shell and utilities give you a usable system. The last phase hardens, measures and documents it, tags version 1.0, and lays out the optional tracks.

Stage V, Ports (Phases 16–18). First you separate the architecture layer: everything x86 moves behind one interface, the generic code learns to compile for other machines, and the kernel learns to read a devicetree instead of Multiboot (16). Then you port Turk-OS to 32-bit RISC-V on QEMU's

`virt` board, with OpenSBI as the firmware, and optionally onto your own Timur-RV32IMC core (17). Finally you port it to 64-bit ARM (AArch64) on QEMU's `virt` board, and optionally to a Raspberry Pi 4. The stage ends with Turk-OS 2.0: the same kernel, userland and TurkFS image on three architectures (18).

5 Phase summary

Phase	Title	You finish with	Weeks	Progress
0	Foundations and toolchain	Cross compiler, QEMU, GDB and a Git repo; C warm-up library with unit tests; verified toolchain sources; threat-model stub	2–3	0 % → 5 %
1	x86 assembly and booting	A 512-byte boot sector; a Multiboot kernel that GRUB and QEMU boot	2–3	5 % → 10 %
2	Getting to C: screen, serial, kprintf	VGA text driver, serial logging, `kprintf`, `panic`	1–2	10 % → 16 %
3	Segmentation, IDT and exceptions	Your own GDT and IDT; register dump on every CPU exception	2	16 % → 22 %
4	Hardware interrupts	PIC, 100 Hz timer, keyboard driver, kernel command monitor	2	22 % → 28 %
5	Physical memory management	Memory map parsing, bitmap then buddy page-frame allocator, first kernel tests	2	28 % → 34 %
6	Paging and the higher-half kernel	Paging on, kernel at `0xC0000000`, VMM API, page-fault reports, read-only kernel code	2–3	34 % → 42 %
7	Kernel heap and data structures	`kmalloc/kfree` on slab caches, intrusive lists, `make test` in headless QEMU, CI and a trap-on-UB build	2	42 % → 48 %
8	Processes and scheduling	Kernel threads, context switch, preemptive round-robin (optional fair scheduler), sleep, FPU state per task	3	48 % → 56 %
9	Synchronization	Spinlocks, wait queues, mutexes, semaphores, `refcount_t`, lock-order checking	2	56 % → 62 %
10	User mode and system calls	TSS, ring 3, `int 0x80`, checked user-memory access, validating ELF loader, threat model, first user programs	3	62 % → 70 %
11	Process API	`fork` (copy-on-write), `exec`, `wait`, `exit`, `brk`, basic signals, system call fuzzer	3	70 % → 77 %
12	Storage	ATA PIO driver, block layer, MBR partitions, buffer cache	2	77 % → 83 %
13	File systems	VFS, file descriptors, tar initrd, writable TurkFS that rejects corrupted images	3–4	83 % → 90 %
14	Userland	Turk-libc, pipes, tty line discipline, `tsh` shell, coreutils	3	90 % → 96 %
15	Hardening and release	Final security audit, users, permissions and sessions, benchmarks, docs, a reproducible and signed v1.0; optional tracks	3–4+	96 % → 100 %
16	Portability: an architecture layer	x86 moved to `kernel/arch/i386`, `include/arch.h`, `struct bootinfo`, a host-tested devicetree parser, one Makefile for three `ARCH` values	2–3	V: 0 % → 20 %

Phase	Title	You finish with	Weeks	Progress
17	The RISC-V port	Turk-OS on `qemu-system-riscv32 -machine virt`: OpenSBI, Sv32, PLIC, virtio-blk, boot to `tsh`; optionally your Timur-RV32IMC core	5–6+	V: 20 % → 60 %
18	The ARM port	Turk-OS on `qemu-system-aarch64 -machine virt`: EL1, GICv2, generic timer, MMU, LP64 userland; v2.0 on three architectures; optionally a Raspberry Pi 4	5–6+	V: 60 % → 100 %

Stage V percentages (marked V) are measured toward Turk-OS 2.0, on their own scale; the 1.0 scale of Phases 0–15 does not change. A + after the weeks means optional work on top of the estimate: the tracks of Phase 15 and the hardware milestones of Phases 17 and 18.

6 Timeline

The bars in Figure 2 place each phase on a calendar at 10–12 hours a week. Stage colors match the phase map. Turk-OS 1.0 is due around week 43 and Turk-OS 2.0 around week 58. Expect Phases 6, 8, 13 and 18 to run over; that is normal, and the rest of the schedule usually absorbs it. The optional hardware milestones of Phases 17 and 18 are not on the calendar.

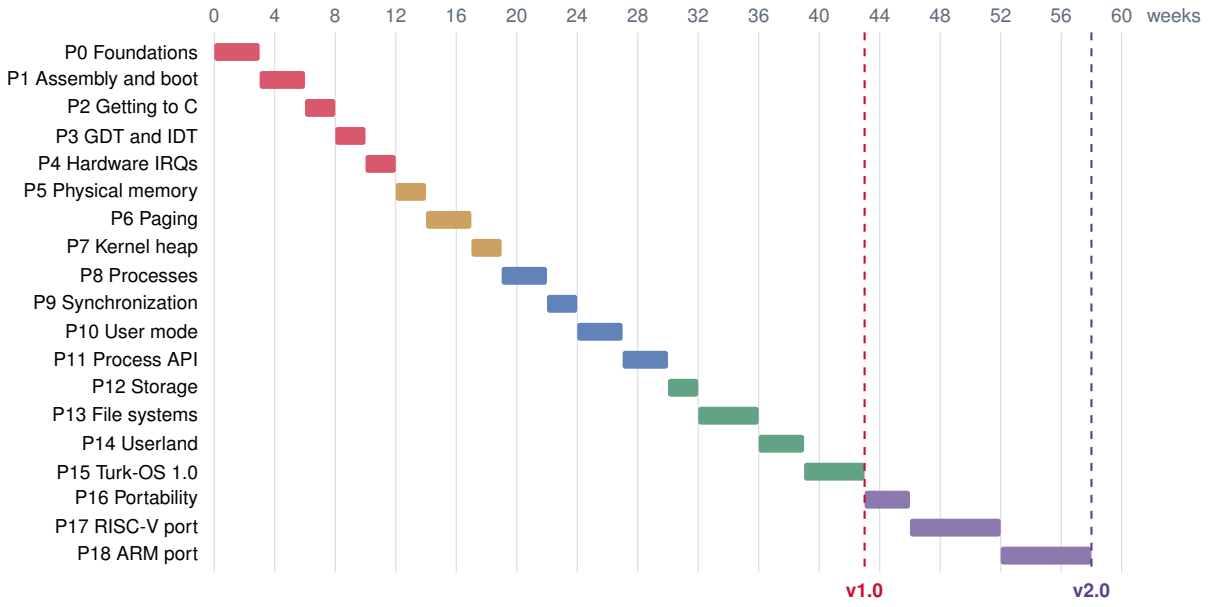


Figure 2: Planned timeline at 10–12 hours per week: about 43 weeks to Turk-OS 1.0 and 58 weeks to Turk-OS 2.0. Treat it as a guide, not a deadline.

7 What Turk-OS 1.0 and 2.0 look like

Figure 3 shows the finished 1.0 system and the phase that builds each part. It is worth coming back to this picture whenever you lose track of where a piece fits. Figure 4 shows what Stage V changes: the generic kernel stays one body of code, and everything that depends on the processor or the board moves below `include/arch.h`, with one backend per architecture.

The source tree grows into the layout below. You don't need to create it all on day one; each phase document says which folders it adds. The entries marked `(Stage V)` arrive in Phases 16–18.

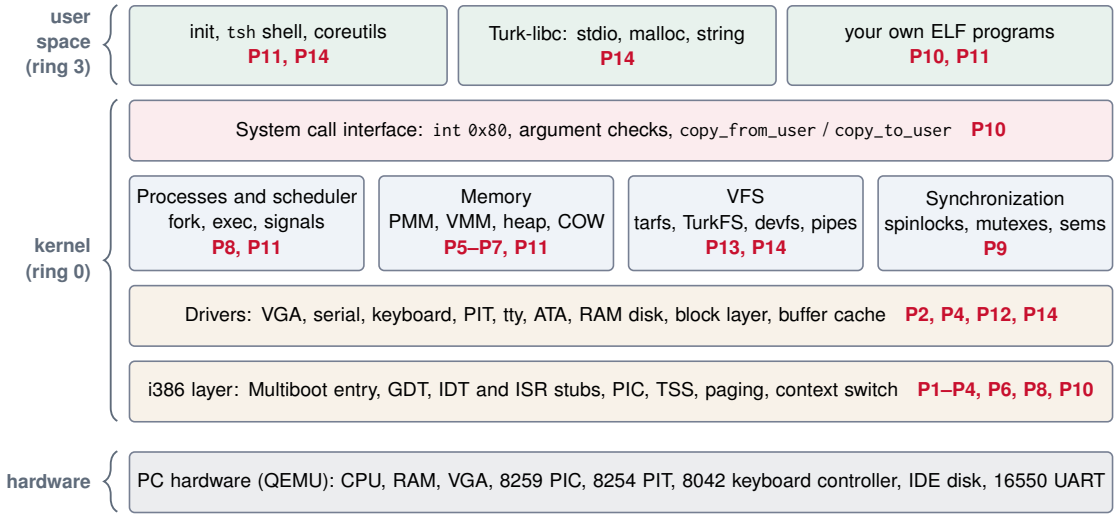


Figure 3: Layers of Turk-OS 1.0, with the phase that builds each part.

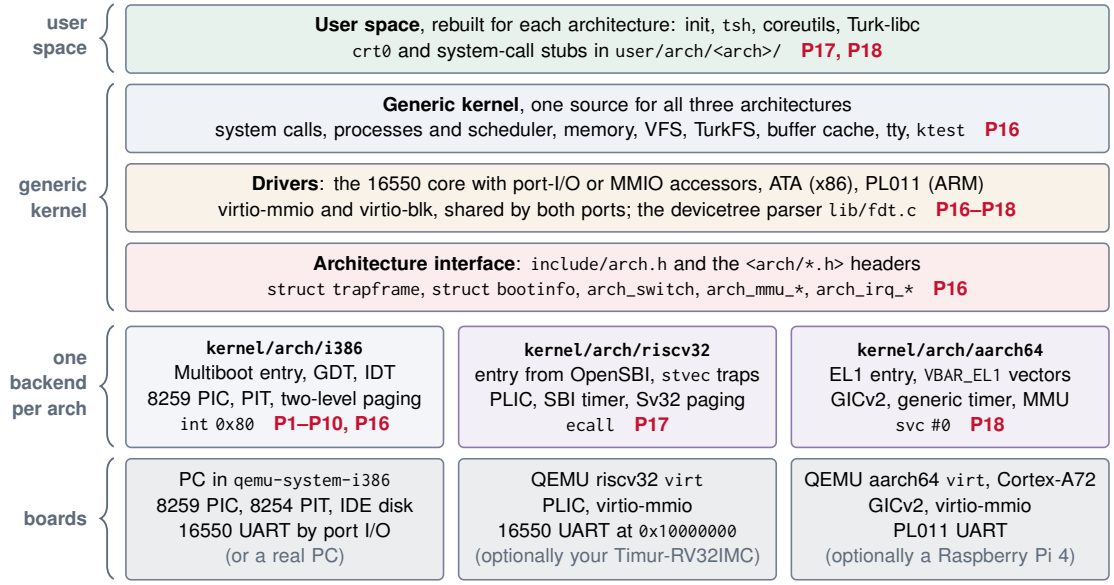


Figure 4: Turk-OS 2.0 after Stage V: one generic kernel, one architecture interface, three backends and three boards.

Turk-OS/ (layout at 1.0, plus the Stage V additions)

```
Turk-OS/
|-- docs/                  this study plan (LaTeX), journal.md, architecture notes
|   `-- porting.md         the leak audit and architecture-interface notes        (Stage V)
|-- boot/                  loader.s (Multiboot entry), grub.cfg
|-- kernel/
|   |-- arch/i386/         gdt.c idt.c isr.s irq.c pic.c pit.c tss.c paging.c switch.s
|   |   `-- include/arch/  the x86 headers behind the architecture interface      (Stage V)
|   |-- arch/riscv32/      boot.S entry.S trap.c sbi.c plic.c timer.c sv32.c      (Stage V)
|   |                      switch.S user.S arch.mk kernel.ld include/arch/
|   |-- arch/aarch64/      boot.S vectors.S trap.c gic.c timer.c mmu.c            (Stage V)
|   |                      switch.S user.S arch.mk kernel.ld include/arch/
|   |-- mm/                pmm.c vmm.c heap.c
|   |-- proc/              task.c sched.c fork.c exec.c signal.c
|   |-- sync/              spinlock.c mutex.c semaphore.c waitq.c
|   |-- fs/                vfs.c file.c tarfs.c turkfs.c devfs.c pipe.c
|   |-- drivers/           vga.c serial.c keyboard.c tty.c ata.c ramdisk.c blockdev.c bcache.c
|   |                      + uart16550.c pl011.c virtio_mmio.c virtio_blk.c       (Stage V)
|   |-- syscall/           syscall.c uaccess.c
|   `-- kmain.c
|-- lib/                   libk: string.c printf.c bitmap.c ringbuf.c list.h
|                          + fdt.c, the devicetree parser                         (Stage V)
|-- include/               kernel headers
|                          + arch.h and bootinfo.h                                (Stage V)
|-- user/
|   |-- libc/              Turk-libc
|   |-- arch/<arch>/       crt0 and system call stubs for each architecture       (Stage V)
|   |-- init/ sh/ bin/     init, tsh, coreutils
|   `-- user.ld
|-- tests/                 ktest suites
|   `-- data/              devicetree blobs dumped from QEMU, for the FDT tests   (Stage V)
|-- tools/                 mkturkfs.c and helper scripts
|-- link.ld
`-- Makefile
```

8 Risks, and what to do when you are stuck

Everyone who writes a kernel hits days where the machine silently reboots and nothing makes sense. The table lists the predictable risks. The protocol after it is how you get unstuck without losing a week.

Risk	Likelihood	Mitigation
Triple faults (instant reboot) with no output	High	Serial logging from Phase 2 on; run QEMU with `-d int,cpu_reset -no-reboot`; GDB; small commits so `git bisect` can find the change that broke it.
Assembly learning curve	High	Phase 1 is dedicated to it. Read compiler output instead of writing assembly from scratch, and keep assembly to tiny stubs.
Memory corruption that shows up far from its cause	High	Assertions everywhere, magic numbers in headers, guard pages, `heap_check()`, deterministic tests.
Stuck on one bug for days	High	Use the protocol below. Time-box the bug, then change approach.
Scope creep (GUI or networking too early)	Medium	Exit criteria are the definition of done. New ideas go on a “later” list for Phase 15.

Risk	Likelihood	Mitigation
A user program or a corrupted disk image can crash the kernel, found only at the end	Medium	Each phase that adds an entry point adds its negative tests at once: evil programs from Phase 10, the system call fuzzer from Phase 11, host fuzzing of every parser from Phase 13, all run in CI.
Toolchain trouble (cross-GCC build, Flatpak sandbox)	Medium	Phase 0 checklist, a host terminal, documented fallback compiler flags.
Outdated instructions in LOB	High	The phase docs give current equivalents (`grub2-mkrescue`, QEMU); the OSDev Wiki has errata.
Losing motivation	Medium	A visible milestone every 2–3 weeks. Record a short demo at each milestone and keep the journal.
Stage V: x86 habits carried into the ports (a forgotten `sfence.vma`, missing TLB maintenance or barriers)	High	Check every step against the specification, single-step in GDB, and compare with xv6-riscv or Linux.
Stage V: QEMU is more forgiving than real hardware (caches, alignment, uninitialised registers)	Medium	Keep real hardware optional and last: the QEMU milestone comes first and counts on its own.
Stage V: toolchain trouble with new targets (multilib, `-march` strings with `_zicsr`)	Medium	Section 9 of `docs/setup.md`, *Toolchains for the ports*, and the Phase 16 checklist (Task 16.1).
Stage V: your own core lacks a feature Turk-OS needs (S-mode, an MMU, a timer)	Medium	Task 17.13's requirements checklist. The QEMU milestone stands on its own without the core.
Stage V: scope creep (SMP, graphics on the ports)	Medium	The Stage V exit criteria are the definition of done.

The getting-unstuck protocol

1. **Reproduce it reliably**, then make it smaller: comment code out until you find the shortest path to the bug.
2. **Look, don't guess.** Use the serial log, `-d int`, the QEMU monitor (`info registers`, `info mem`), and GDB breakpoints.
3. **Write the hypothesis down** in your journal before you change code. Change one thing at a time.
4. **Re-read the section** of the Intel manual (in Stage V, the RISC-V or Arm specification) or the book chapter for the exact hardware rule involved.
5. **Compare with MINIX** (OSDI source) or xv6: how does a working kernel do the same step?
6. **After two sessions without progress**, ask for help (OSDev forum, a classmate, your advisor) and include your journal notes. Explaining the problem often solves it.

9 Master milestone tracker

Print this page and tick each box when that phase's exit criteria are fully met.

☐ **P0** `i686-elf-gcc`, NASM, QEMU and GDB work; repo created; toolchain signatures verified; C warm-up library and overflow helpers pass their tests; threat-model stub written.

☐ **P1** Boot sector prints a message; Multiboot kernel boots via `-kernel` and via a GRUB ISO.

☐ **P2** Colored banner on screen, same log on the serial port, `kprintf` and `panic` tested; the kernel builds with `-Werror`.

☐ **P3** Own GDT and IDT loaded; divide-by-zero shows a register dump; `int3` returns cleanly.

☐ **P4** Timer ticks at 100 Hz; keyboard input with shift and caps; monitor commands run.

☐ **P5** Memory map printed; bitmap and then buddy allocator pass the tests from 16 MiB to 3 GiB of RAM.

☐ **P6** Kernel runs at `0xC0000000`; null dereference produces a clean page-fault report; a write to kernel code faults.

☐ **P7** 10 000-operation stress test passes through the slab caches; `make test` runs headless and in CI; the trap-on-UB build passes; allocation failures are handled.

☐ **P8** Several kernel threads preempt each other; `sleep` works; no leaks over 1000 task cycles; FPU state stays with its task.

☐ **P9** Race demo fixed by locks; bounded buffer and dining philosophers run without deadlock; lock order checked.

☐ **P10** Ring-3 program prints through `write`; faulty and malicious programs are killed or get errors, the kernel survives; malformed ELF files are rejected; every system call has a row in the boundary table.

☐ **P11** `fork/exec/wait` chain works; copy-on-write verified, also through `copy_to_user`; no leaks over 1000 forks; ten minutes of fuzzing without a panic.

☐ **P12** QEMU disk identified; data survives reboot; partitions detected and corrupted tables rejected; buffer cache in use.

☐ **P13** Files written by Turk-OS pass `fsck.minix` on the host; file descriptor semantics tested; corrupted images are rejected without a panic; the parsers survive an hour of host fuzzing.

☐ **P14** Boots to the `tsh` prompt; pipes, redirection and Ctrl+C work; coreutils present; resource limits enforced.

☐ **P15** One hour of fuzzing as an ordinary user without a panic; permission and session tests pass; benchmarks, docs and the threat model finished; two clean builds match and v1.0 is tagged and signed.

☐ **P16** Generic code free of x86 (audit in `docs/porting.md`); `make ARCH=i386 test` passes; generic code compiles for riscv32 and aarch64 without warnings; FDT parser passes its host tests.

☐ **P17** `qemu-system-riscv32 -machine virt` boots to the `tsh` prompt; `make ARCH=riscv32 test` passes, evil programs included; W^X holds; a TurkFS image written on RISC-V passes `fsck.minix`.

☐ **P18** `qemu-system-aarch64 -machine virt` boots to the `tsh` prompt; `make ARCH=aarch64 test` passes, evil programs included; W^X holds; the same TurkFS image works on all three; v2.0 tagged.

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. LOB and OSTEP are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.

Port references used in Stage V

XV6 Russ Cox, Frans Kaashoek, Robert Morris. *xv6: a simple, Unix-like teaching operating system*, RISC-V edition. The companion book to MIT's xv6-riscv kernel. Free at https://pdos.csail.mit.edu/6.1810/; source at https://github.com/mit-pdos/xv6-riscv.

RISCV *The RISC-V Instruction Set Manual*, Volume I (Unprivileged ISA) and Volume II (Privileged Architecture); *RISC-V Supervisor Binary Interface Specification*. Free at https://riscv.org/specifications/ratified/ and https://github.com/riscv-non-isa/riscv-sbi-doc.

ARM *Arm Architecture Reference Manual for A-profile architecture* (DDI 0487), and Arm's *Learn the architecture* guides: AArch64 exception model, AArch64 memory management, Generic Timer, GIC. Free at https://developer.arm.com/documentation.

SPEC *Devicetree Specification* (https://www.devicetree.org/specifications/); *Virtual I/O Device (VIRTIO)*, OASIS (https://docs.oasis-open.org/virtio/virtio/); the QEMU documentation for the RISC-V and Arm `virt` boards (https://www.qemu.org/docs/master/system/).

All four are free. Specifications are revised often, so the reading plans cite them by chapter and section title rather than by page.